

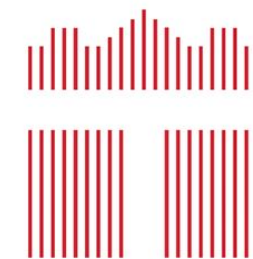
Text Preparation in R and Python: Collaborative Workflow

QTA principles, cleaning, tokens, and
document-feature matrices

Yen-Chieh Liao 廖彥傑

2026-09-18

Last compiled: 2026-09-18 11:19:03 (Asia/Taipei)



**National
Taiwan
University**
國立臺灣大學

Learning objectives

Week 2: Text Preparation in R and Python: Collaborative Workflow

- Set up RStudio and Positron for text analysis with R and Python.
- Organize code with GitHub and submit written coursework through shared Overleaf projects.
- Inspect supplied corpora and check project data feasibility: coverage, metadata, and access.
- Clean and tokenize text, build a DFM, and evaluate preprocessing choices.

Week 4 content introduced today: Text preprocessing and document-feature matrices (DFMs). Week 4 builds on these with dictionaries, sentiment analysis, and validation.

Five principles of quantitative text analysis

1. **Domain knowledge** defines the research problem and relevant evidence.
2. **Human judgment** guides interpretation and validation.
3. **Discovery and testing** require distinct stages.
4. **The task** determines which methods are useful.
5. **Validation** evaluates whether the measure fits the claim.

Selecting documents: Chapter 4

(Grimmer et al. 2022)

Classroom example: measuring expressed preference for coffee.

Design choice	Example
Target population	Thai-language public posts about coffee in a specified month
Quantity of interest	Share of those posts expressing positive preference for coffee
Observed corpus	Posts returned by one platform API and a specified keyword search

Found data: document how the texts were produced, preserved and retrieved.

The next step is to examine what the collection process leaves out.

Selection bias begins before preprocessing

Source of bias	Example	Research consequence
Resources	Well-funded groups archive more documents	Available texts overrepresent some actors
Incentives	Politicians publish strategically	Public statements differ from private beliefs
Medium	A platform limits length or visibility	The medium shapes expression
Retrieval	Searching only for “economy”	Relevant texts with other words disappear

Corpus boundaries and document units

Decision	Example	What to record
Population	Parliamentary speech contributions	Chamber, actors, dates
Inclusion	All contributions in selected debates	Rule and exclusions
Document unit	One contribution per row	How segments were defined
Identifier	A persistent <code>doc_id</code>	Links to source and metadata

Changing the unit to speaker-day or party-year changes the comparison.

Sources and import routes

Source	Typical route	First check
Existing corpus	Download plus codebook	Coverage and sampling
TXT, CSV, TSV	<code>readr</code> , <code>data.table</code> , base R	Encoding and columns
JSON or API	<code>jsonlite</code> , <code>httr2</code>	Pagination and identifiers
HTML or XML	<code>rvest</code> , <code>xml2</code>	Boilerplate and structure
PDF or DOCX	<code>pdftools</code> , <code>readtext</code>	Reading order and document boundaries

Scanned PDFs may need OCR before they contain usable text.

The R text-analysis toolkit in 2017

Table 1. An overview of text analysis operations, with the R packages used in this Teacher’s Corner.

Operation	R packages	
	example	alternatives
Data preparation		
importing text	<i>readtext</i>	<i>jsonlite, XML, antiword, readxl, pdftools</i>
string operations	<i>stringi</i>	<i>stringr</i>
preprocessing	<i>quanteda</i>	<i>stringi, tokenizers, snowballC, tm, etc.</i>
document-term matrix (DTM)	<i>quanteda</i>	<i>tm, tidytext, Matrix</i>
filtering and weighting	<i>quanteda</i>	<i>tm, tidytext, Matrix</i>
Analysis		
dictionary	<i>quanteda</i>	<i>tm, tidytext, koRpus, corpustools</i>
supervised machine learning	<i>quanteda</i>	<i>RTextTools, kerasR, austin</i>
unsupervised machine learning	<i>topicmodels</i>	<i>quanteda, stm, austin, text2vec</i>
text statistics	<i>quanteda</i>	<i>koRpus, corpustools, textreuse</i>
Advanced topics		
advanced NLP	<i>spacyr</i>	<i>coreNLP, cleanNLP, koRpus</i>
word positions and syntax	<i>corpustools</i>	<i>quanteda, tidytext, koRpus</i>

Historical package overview (2017). Package responsibilities and APIs have since changed.

Beyond 2017: annotation and embeddings

Tool / access from R	Model type	Output and use
UDPipe 1: R <code>udpipe</code>	Supervised annotation pipeline trained on a treebank	Tokens, lemmas, POS, dependency labels
UDPipe 2: R + REST API	Neural annotation with contextual embeddings	Linguistic annotations; current models use BERT-based representations
Flair: R <code>reticulate</code> + Python (<code>flairR</code> ; Liao et al., 2023)	Contextual string embeddings or Transformer embeddings	Word/document vectors; models for NER, tagging and classification

FlairEmbeddings uses character language models; **TransformerWordEmbeddings** uses models such as BERT.

R can prepare text, call these tools, and analyze the returned annotations or vectors.

quanteda, UDPipe, spaCy, and Flair

Tool / access	Main emphasis	Typical output
quanteda (R)	Corpus management and feature analysis	Corpus, tokens, DFM; KWIC and TF-IDF
UDPipe (R / API)	Linguistic annotation using trained models	Tokens, lemmas, POS, UD dependency labels
spaCy (Python)	Configurable NLP pipelines	Tokens, POS, dependencies, named entities
Flair (Python) flaiR: R wrapper for Python Flair	Embeddings and neural task models	Vectors; entity labels, POS, text categories

The tools can work together: lemmas or selected annotations can become features in a quanteda DFM. Capabilities overlap and depend on the language, model, and task.

Beyond 2017: R and Python in one workflow

One possible application: compare the meanings of policy statements.

Stage	Example tools	Contribution
Prepare in R	<code>stringr</code> , <code>quanteda</code>	Clean text and preserve document IDs
Represent in Python	<code>sentence-transformers</code>	Encode texts as semantic vectors
Evaluate in R	Statistical models and plots	Validate similarities and interpret patterns

The languages can share a project. Each stage follows the research question.

Connecting R and Python

Python inside an R session

- R's **reticulate** package imports Python modules and calls their functions.
- I frequently used **reticulate** during my PhD (2018–2022).
- It converts supported objects between R and Python.
- Record the Python environment as well as R packages.

Separate scripts, shared data

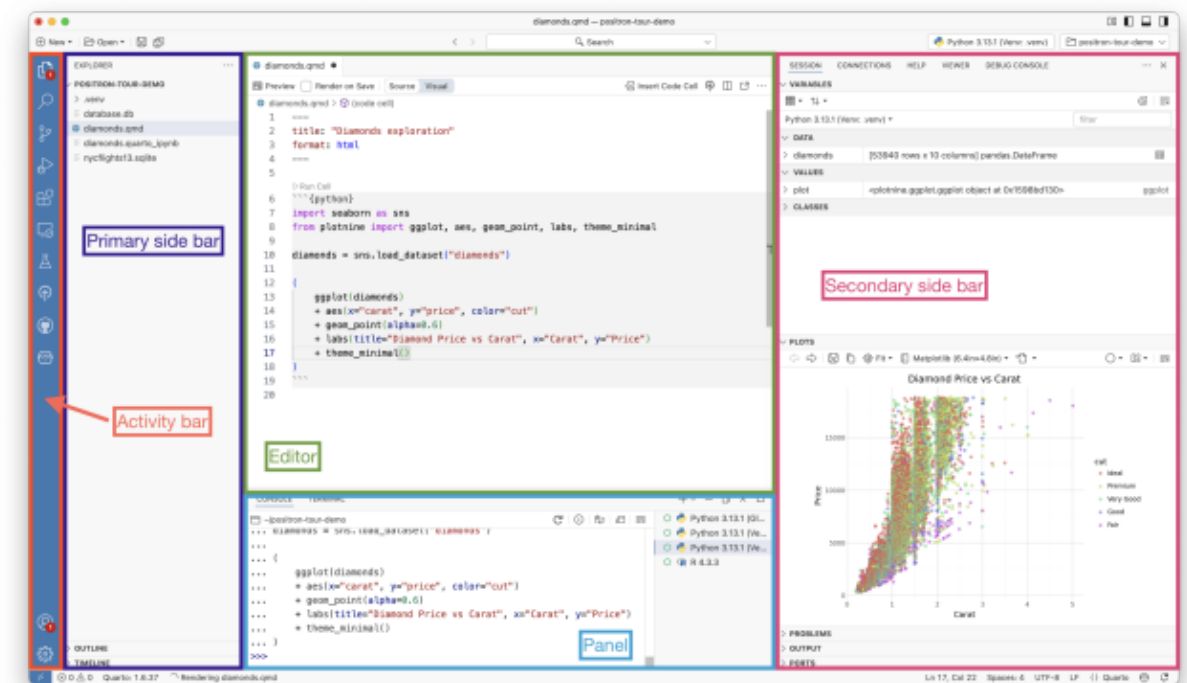
- R and Python scripts exchange files such as CSV or Parquet.
- A stable **document ID** connects inputs and outputs.
- Check row order, encoding and data types at each handoff.

An IDE organizes the work. Object or file exchange connects the languages.

Positron: one IDE for R and Python

In 2022, RStudio the company became Posit, reflecting its broader support for data science across languages.

- Posit developed **Positron** for multilingual data science.
- R and Python have built-in support, with consoles and data exploration.
- The **RStudio IDE** remains a separate, maintained product.



Our classroom examples use R.

The sequence of text-analysis operations

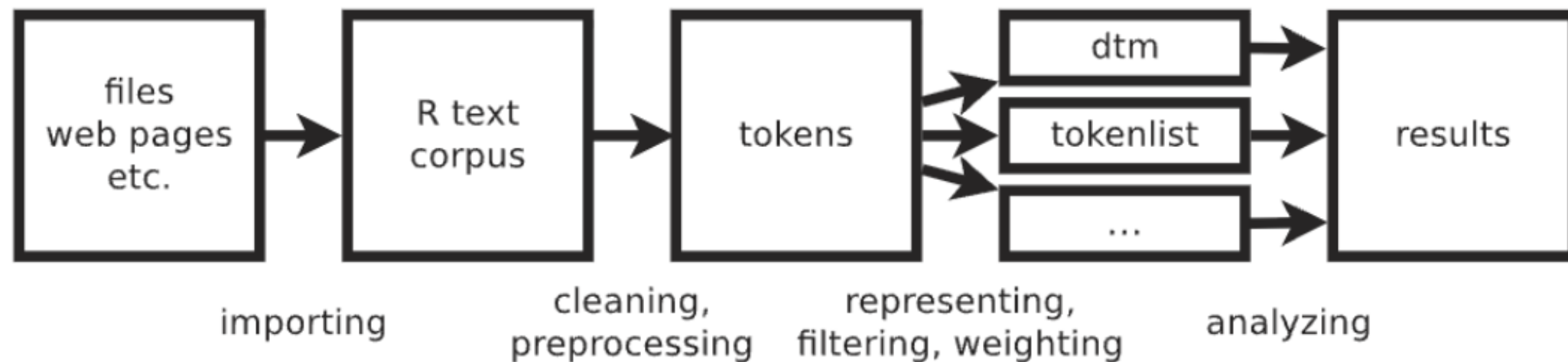


Figure 1. Order of text analysis operations for data preparation and analysis.

The branch matters: analysis can use a matrix, token positions, or other representations.

Objects used in this session

Object	Contents	Use
Data frame	Text, identifiers, metadata	Import checks and joins
<code>corpus</code>	Text with document variables	Subsets and metadata access
<code>tokens</code>	Sequences of tokens	Phrase matching and context inspection
<code>dfm</code>	Documents × features	Counts and weighted representations

`stringr` handles strings. `quanteda` provides the corpus, tokens, and DFM workflow.

Import checks and encoding

```
texts <- readLines("data/speeches.txt", encoding = "UTF-8")  
length(texts)  
head(texts)
```

- Confirm the source encoding rather than guessing from appearance.
- Inspect accented characters and Chinese text after import.
- Count missing texts and duplicated identifiers.
- Check whether a row is a line, paragraph, or whole document.

A UTF-8 argument does not repair a wrongly identified source encoding.

A parliamentary example

Field	Value in the source slide
Date	1997-01-20
Speaker	William Hague
Party	Conservative
Agenda	Public Expenditure [Oral Answers To Questions > Wales]

Text includes [Interruption.], **DRAMATICALLY**, and !!!.

Which elements are metadata, meaningful language, or recording conventions?

Cleaning depends on the research question

Element	Possible treatment	Information at stake
Speaker and party	Store as metadata	Who speaks
[<i>Interruption.</i>]	Remove or extract into an indicator	Parliamentary interaction
Capitalization	Preserve or lowercase	Names and emphasis
Numbers	Keep, normalize, or remove	Years, budgets, percentages
Repeated punctuation	Preserve or simplify	Rhetorical emphasis

Keep an untouched copy of the raw text.

String operations with stringr

Function	Purpose	Example use
<code>str_detect()</code>	Test whether a pattern appears	Flag bracketed annotations
<code>str_extract()</code>	Retrieve the first match	Extract a year
<code>str_remove_all()</code>	Remove all matches	Delete selected annotations
<code>str_replace_all()</code>	Replace matches	Standardize separators
<code>str_squish()</code>	Normalize whitespace	Remove repeated spaces
<code>str_to_lower()</code>	Change case	Merge case variants

Bracket patterns and unintended deletion

```
s <- "Trade [Commons] and jobs [Wales]"

str_remove_all(s, "\\[.*\\]")
# "Trade "

str_remove_all(s, "\\[[^\\[\\]]*\\]") |> str_squish()
# "Trade and jobs"
```

The first pattern spans from the first `[` to the last `]`. The second handles separate, non-nested bracket groups.

A cleaning example with an audit trail

```
agenda_raw <- "  Export Trade [Commons] -- vol. 407  "  
agenda_clean <- agenda_raw |>  
  str_remove_all("\\[[^\\[\\]]*\\]") |>  
  str_to_lower() |>  
  str_replace_all("[^[:alnum:][:space:]]", " ") |>  
  str_squish()  
agenda_clean
```

```
[1] "export trade vol 407"
```

This rule preserves numbers and removes punctuation. Another task may need a different rule.

A worked example: a small text corpus

ID	Synthetic text
d1	Health workers need health services.
d2	The government supports health services.
d3	Workers do not support the policy.
d4	The government supports the policy.

These short texts let us inspect every token and verify every count.

Creating a corpus

```
library(quanteda)

corp <- corpus(
  raw,
  docid_field = "doc_id",
  text_field = "text"
)

docnames(corp)
docvars(corp)
```

Each row supplies one text. The remaining columns become document variables.

Tokens, types, and vocabulary

Text: “Health services need health workers.”

Representation	Tokens	Distinct types
Case and punctuation retained	6	6
Lowercase, punctuation removed	5	4

After normalization: `health, services, need, health, workers`

A **token** is an occurrence. A **type** is a distinct form under the chosen representation.

Tokens in R text analysis and LLMs

Both use **tokens**: units produced by tokenization for analysis or model processing.

Comparison	Common R text-analysis workflow	Typical LLM workflow
Common units	Words; optionally characters or sentences	Often subwords; can include whole words, characters or byte units
Segmentation depends on	Package, tokenizer and your settings	The model's tokenizer and vocabulary
Main use	Word counts, DFM, TF-IDF, topic models	Token IDs used for model input and text generation

One word can become several LLM tokens. R can call either kind of tokenizer; the workflow determines the unit.

Tokenizing with explicit choices

```
toks <- tokens(corp, remove_punct = TRUE) |>  
  tokens_tolower()  
  
as.list(toks)[[1]]
```

```
[1] "health"    "workers"   "need"      "health"    "services"
```

Inspect the output when texts contain hyphens, URLs, abbreviations, or non-Latin scripts.

Preprocessing choices and information loss

Choice	Possible benefit	Possible loss
Lowercase	Merge <code>Health</code> and <code>health</code>	Proper-name distinctions
Remove stopwords	Reduce frequent function words	Negation or writing style
Stem or lemmatize	Merge some word forms	Differences in meaning
Trim rare features	Reduce vocabulary size	Rare but important concepts

The relevant question is how the choice affects the intended measure.

Stopwords and negation

```
sw <- setdiff(stopwords("en"), c("not", "no", "nor"))
toks_content <- tokens_remove(toks, pattern = sw)
```

Rule applied to document d3	Remaining tokens
Standard English stopword list	workers support policy
List that retains negation	workers not support policy

Retaining `not` preserves a clue. A unigram model still needs a way to use its scope.

The bag-of-words representation

A DFM stores **one row per document** and **one column per feature**.

ID	health	workers	government	policy	not
d1	2	1	0	0	0
d2	1	0	1	0	0
d3	0	1	0	1	1
d4	0	0	1	1	0

Selected columns from our count DFM. Features can also represent phrases or other categories.

What word counts cannot recover

“Workers support government.”

“Government support workers.”

The two sentences have the same lowercase unigram counts.

- Word order indicates who supports whom.
- Negation requires relationships between words.
- Ambiguous words need context.
- Sarcasm may depend on discourse beyond a sentence.

N-grams and syntax retain some additional information.

Multiword expressions

```
phrase_toks <- tokens(  
  "We support the armed forces.",  
  remove_punct = TRUE  
) |> tokens_tolower()  
  
tokens_compound(  
  phrase_toks,  
  pattern = phrase("armed forces")  
)
```

`armed_forces` becomes a single feature. Detect phrases before removing words that belong inside them.

Bigrams and operation order

```
tokens("Workers do not support the policy.",  
      remove_punct = TRUE) |>  
  tokens_tolower() |>  
  tokens_ngrams(n = 2)
```

workers_do, do_not, not_support, support_the, the_policy

Generate these before stopwords removal if original adjacency matters. Deleting words first can create pairs that were never adjacent.

Collocations and grammatical features

Feature strategy	Example	Required judgment
Known phrase	<code>armed_forces</code>	Which phrases matter?
Statistical collocation	Candidate such as <code>northern_ireland</code>	Is association substantively useful?
POS selection	Nouns, verbs, adjectives	Which grammatical roles fit the task?

A frequent or associated word pair is a candidate feature. Inspect its uses before adding it to the measurement rule.

Stemming and lemmatization

Method	Example	Limitation
Stemming	<code>studies</code> becomes <code>studi</code>	Stems may not be dictionary words
Lemmatization	<code>studies</code> may become <code>study</code>	Accuracy depends on language and context
Explicit replacement	Map <code>forces</code> to <code>force</code>	Only the supplied mapping is handled

Neither approach guarantees correct semantic grouping. A dictionary built from full words may fail to match stemmed features.

Language and model requirements

Language

- English whitespace is not a universal word boundary.
- Chinese segmentation needs inspection.
- Stopword and morphology tools are language specific.

Model

- A DFM may use word or phrase features.
- A pretrained model expects its own tokenizer.
- Preserve the input distinctions that model requires.

A BoW preprocessing recipe should not automatically become an LLM input recipe.

UDPipe models: Japanese, Chinese, and Thai

Input language	Access from R	Model
Japanese	<code>udpipe</code> : local UDPipe 1	<code>japanese-gsd</code> (UD 2.5)
Traditional Chinese	<code>udpipe</code> : local UDPipe 1	<code>chinese-gsd</code> (UD 2.5)
Thai	<code>httr2</code> : UDPipe 2 API	<code>thai-tud</code> (UD 2.17)

- All three provide segmented `token` and predicted `upos` (POS).
- Dependency parsing adds heads and relation labels.
- `lemma` is a predicted base form; this Thai model returns `_` (unavailable).

Instructor demonstration: interpret the prepared model outputs shown in these slides.

Annotating UTF-8 text with UDPipe

```
library(udpipe)
# Set model_path to a downloaded Chinese-GSD model.
zh_model <- udpipe_load_model(model_path)
ann_zh <- udpipe_annotate(zh_model, x = "我喜歡咖啡。",
  doc_id = "zh_coffee", parser = "default") |> as.data.frame()
ann_zh[c("token_id", "token", "upos",
  "head_token_id", "dep_rel")]
```

Chinese: `chinese-gsd`; English: `english-ewt`; Thai: R + UDPipe 2 API.

- `parser = "default"` adds dependency parsing to tokens and POS.
- `head_token_id` identifies the head; `dep_rel` labels the relationship. Head 0 marks the sentence root.

Word tokenization across languages

Same meaning: "I like coffee."

Thai

R: `httr2 + UDPipe 2 API`

ฉันชอบกาแฟ

Tokens (POS)

ฉัน (PRON)

ชอบ (VERB)

กาแฟ (NOUN)

3 tokens

The Thai model finds
word boundaries.

POS = part of speech. Counts exclude punctuation.

Japanese

R: `udpipe (japanese-gsd)`

私はコーヒーが好きです。

Tokens (POS)

私 (PRON) / は (ADP)

コーヒー (NOUN) / が (ADP)

好き (ADJ) / です (AUX)

6 tokens

Particles and auxiliaries
remain separate tokens.

Chinese

R: `udpipe (chinese-gsd)`

我喜歡咖啡。

Tokens (POS)

我 (PRON)

喜歡 (VERB)

咖啡 (NOUN)

3 tokens

Multi-character words can
form single tokens.

Dependency labels across languages

Same meaning: "I like coffee."

Chinese (Traditional)		English		Thai	
R: udpipe (chinese-gsd)		R: udpipe (english-ewt)		R: httr2 + UDPipe 2 API	
我喜歡咖啡。		I like coffee.		ฉันชอบกาแฟ	
Token (POS)	dep_rel	Token (POS)	dep_rel	Token (POS)	dep_rel
1 我 (PRON)	nsubj	1 I (PRON)	nsubj	1 ฉัน (PRON)	nsubj
2 喜歡 (VERB)	root	2 like (VERB)	root	2 ชอบ (VERB)	root
3 咖啡 (NOUN)	obj	3 coffee (NOUN)	obj	3 กาแฟ (NOUN)	obj
Head IDs: 2 / 0 / 2		Head IDs: 2 / 0 / 2		Head IDs: 2 / 0 / 2	

nsubj = nominal subject; root = sentence root; obj = object.

Head 0 marks the root; other IDs point to a token in the same sentence.

Punctuation is omitted from this display.

Japanese: observed segmentation

政府は医療従事者を支援する。

ID	token	lemma	upos	head	dep_rel
1	政府	政府	NOUN	6	nsubj
2	は	は	ADP	1	case
3	医療	医療	NOUN	4	compound
4	従事者	従事者	NOUN	6	obj
5	を	を	ADP	4	case
6	支援	支援	VERB	0	root
7	する	する	AUX	6	aux
8	。	。	PUNCT	6	punct

Dependency parsing: `head` = head token ID (0 = root); `dep_rel` = relation.

Traditional Chinese: observed segmentation

政府支持醫療工作者。

ID	token	lemma	upos	head	dep_rel
1	政府	政府	NOUN	5	nmod
2	支持	支持	NOUN	5	nmod
3	醫療	醫療	NOUN	5	nmod
4	工作	工作	VERB	5	compound
5	者	者	PART	0	root
6	。	。	PUNCT	5	punct

Dependency parsing: `head` = head token ID (0 = root); `dep_rel` = relation.

Model errors: 支持 is tagged `NOUN`; 者 is predicted as `root`. Inspect the original sentence before interpreting these outputs.

Negation and POS-based feature selection

政府不支持刪減醫療預算。

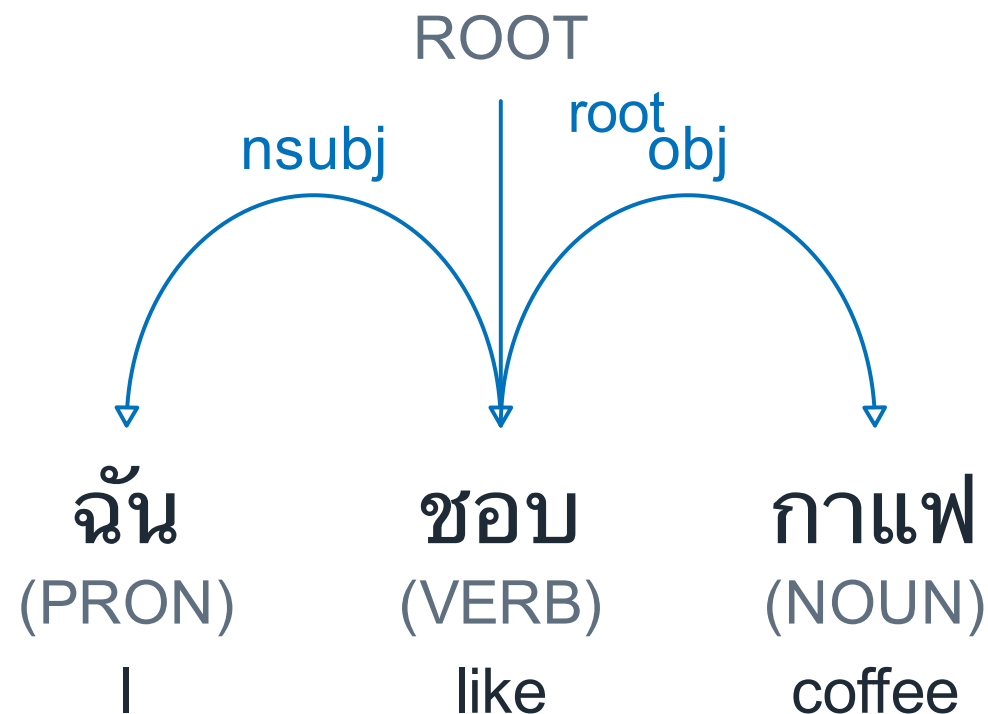
ID	token	lemma	upos	head	dep_rel
1	政府	政府	NOUN	3	nsubj
2	不	不	ADV	3	advmod
3	支持	支持	VERB	0	root
4	刪減	刪減	VERB	3	xcomp
5	醫療	醫療	NOUN	6	nmod
6	預算	預算	NOUN	4	obj
7	。	。	PUNCT	3	punct

Dependency parsing: `head` = head token ID (0 = root); `dep_rel` = relation.
Keeping only `NOUN` and `VERB` removes 不 (`ADV`), attached to 支持 (ID 3) by `advmod`. The feature list loses negation.

From Thai syntax to a research measure

ฉันชอบกาแฟ "I like coffee."

Dependency relations



Arrows run from head to dependent.

Corpus measure: share of documents mentioning a target that express positive preference.

- **Language relationships**

POS marks word classes. Dependencies link a subject to a predicate and target.

- **Operationalize preference**

Link the subject to "like" and its object.
Check that the statement is affirmative.

- **Code this example**

Coffee: positive preference = 1.

- **Research question**

Which targets receive more expressions of positive preference?

UDPipe annotations as quanteda tokens

```
# Assume ann_multi contains an annotation table.
a <- subset(ann_multi, upos != "PUNCT")
a <- a[order(a$doc_id, a$sentence_id,
            as.integer(a$token_id)), ]

ja_tokens <- split(a$token[a$doc_id == "ja_01"],
                  a$doc_id[a$doc_id == "ja_01"])
ja_dfm <- dfm(as.tokens(ja_tokens), tolower = FALSE)
```

Use `as.tokens()` to preserve the existing segmentation. Build the Chinese DFM from the Chinese documents in the same way.

Code illustration: annotations → tokens → DFM.

Tokens become DFM features

Same tokens; illustrative POS tags in parentheses

A workers (NOUN) / support (VERB) / reform (NOUN)

B workers (NOUN) / oppose (VERB) / reform (NOUN)

1 Count word tokens

Keep all word types.

dfm(toks)

2 Select tokens by POS

Keep NOUN: workers, reform.

Filter tokens, then dfm().

3 Map tokens with a dictionary

PRO: support, endorse

CON: oppose, reject

tokens_lookup(), then dfm().

Word columns

doc	workers	support	reform	oppose
A	1	1	1	0
B	1	0	1	1

Word columns after POS selection

doc	workers	reform
A	1	1
B	1	1

Category columns

doc	PRO	CON
A	1	0
B	0	1

Rows = documents; columns = chosen features; cells = counts.

Noun filtering loses the stance contrast; dictionary counts still need validation.

Building and inspecting the DFM

```
x <- dfm(toks_content)
dim(x)
topfeatures(x, 6)
```

```
[1] 4 9
```

health	workers	services	government	supports	policy	
	3	2	2	2	2	2

Rows retain the document IDs. The full DFM includes features beyond the selected columns shown earlier.

Term frequency and document frequency

Feature	Total occurrences	Documents containing it
health	3	2
workers	2	2
not	1	1

```
docfreq(x)
x_trim <- dfm_trim(x, min_docfreq = 2,
                  docfreq_type = "count")
```

This threshold removes `not`. A rare feature can still carry essential meaning.

Counts, binary indicators, and TF-IDF

Representation	Meaning of a cell	Possible use
Count	Number of occurrences	Frequency-based measurement
Binary	Whether the feature appears	Presence-based comparison
TF-IDF	Count adjusted for corpus prevalence	Document retrieval or comparison

The choice changes what repeated words contribute. Model requirements may constrain which representation is appropriate.

TF-IDF with a stated formula

For this lesson, use raw counts and a base-10 logarithm:

$$\text{TF-IDF}(d, t) = \text{count}(d, t) \times \log_{10}(N / \text{df}(t))$$

- **N** is the number of documents in the reference corpus.
- **df(t)** counts documents that contain feature *t*.
- A feature in every document receives $\text{IDF} = 0$ under this formula.

Different normalizations and smoothing rules produce different weights.

TF-IDF in the worked example

For `health`, $N = 4$, $df = 2$, so $IDF = \log_{10}(2) \approx 0.3010$.

Document	Count of <code>health</code>	TF-IDF
d1	2	0.6021
d2	1	0.3010
d3	0	0
d4	0	0

```
x_weighted <- dfm_tfidf(x, scheme_tf = "count",
                        scheme_df = "inverse", base = 10)
```

Context checks with KWIC

```
kwic(toks, pattern = "support*", window = 3)
```

docname	pre	keyword	post
d2	the government	supports	health services
d3	workers do not	support	the policy
d4	the government	supports	the policy

Read the context around a matched term before treating its count as support for a policy.

Wrap-up: DFM, KWIC, TF-IDF

Tool	What it lets you do
DFM	Represent document features for comparison and modeling.
KWIC	Read terms in context and check what feature counts mean.
TF-IDF	Emphasize distinctive terms for retrieval and similarity.

Wordfish is an unsupervised method for scaling texts on **one dimension**.

Word-count DFM → e.g., Wordfish → relative document positions

Use: compare party manifestos or speeches. **Input:** counts, not TF-IDF weights.
Interpret and validate the scale before calling it ideology.

Sensitivity to preprocessing

Compare	Inspect	Explain
With / without stopwords	Negation and grammatical context	Which distinctions vanish?
Unigrams / phrases	Compound concepts	Which matches change?
Full / trimmed vocabulary	Removed features and empty rows	Which documents lose evidence?
Counts / TF-IDF	Relative feature contributions	Does the result depend on weighting?

Keep the corpus and research question fixed while comparing a chosen rule.

In-class setup: install, open, run

Download and install: [R](#) and [Python 3](#), then [RStudio Desktop](#) and [Positron](#).

Try the consoles: `1 + 1` in R; `print(1 + 1)` in Python. Both return 2.

Open `Lab_Session_W2.R` in `lab/`. In the **R console**, install:

```
install.packages(c("dplyr", "stringr", "ggplot2",  
  "quanteda", "quanteda.textstats", "quanteda.textplots"))
```

Set the working directory to `W2/lab/`. Keep its `Data/` folder alongside the script. Run the lab section by section at home.

Take-home skill: Open an `.R` file in RStudio or Positron and run the lab independently.

Take-home lab

Complete independently at home. Use the supplied sample of 5,000 House of Commons speech contributions (1945–2025).

1. Inspect strings and clean agenda metadata with `stringr`.
2. Build, subset, and reshape a `quanteda` corpus.
3. Tokenize, preserve phrases, and inspect KWIC contexts.
4. Build and trim DFMs; explore frequencies, plots, and keyness.
5. Work through the **11 practice exercises** in the lab.

Folder: `lab/`

Open `Lab_Session_W2.qmd` or its `.R` companion. Data are included.

Take-home practice and support

- Run the worked examples **section by section** and inspect each output.
- Attempt all **11 exercises** and save your code and questions.
- Bring the exercise number, relevant code, and output or error message.
- Discuss questions **before class, if time permits**, or ask during **TA hours**.

Be ready to explain what your code changes and how that affects the evidence.

Discussion and answer guide

Why can deleting a common word damage a measure?

A word such as `not` can determine the meaning of the surrounding statement.

Why does document frequency change after aggregation?

Its value depends on what counts as one document.

Does successful preprocessing validate a substantive claim?

It validates an implementation only in limited ways. The measure still needs evidence appropriate to the research question.

Readings and software references

Primary lecture source

Schoonvelde, M. (2026). *Quantitative text analysis – Essex Summer School: Principles of QTA, cleaning text, and preprocessing text*. Supplied slides, 38 pages.

Reading and reproduced figure/table

Welbers, K., Van Attevelde, W., & Benoit, K. (2017). *Text Analysis in R*. Communication Methods and Measures, 11(4), 245–265. [doi:10.1080/19312458.2017.1387238](https://doi.org/10.1080/19312458.2017.1387238).

Corpus selection reading

Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as Data: A New Framework for Machine Learning and the Social Sciences*. Princeton University Press. Chapter 4: “Selecting Documents.” [Book](#).

Software documentation

[quanteda: tokens, TF-IDF, KWIC, stringr: pattern removal](#).

[UDPipe guide, Japanese GSD, Chinese GSD](#).

The English, Thai, Japanese, and Chinese texts are synthetic teaching examples.